

Codebase Change Summary — Week 4

Test-Driven Development, Mocking, and Dependency Isolation

SE 413/513 — Software Engineering

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 3, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time rather than being rebuilt from scratch each week. This document summarizes exactly what was added or changed for Week 4 and, more importantly, explains *why* each change exists — what testing concept from the Week 4 reading it is meant to demonstrate. Read this alongside the reading and `demo-testing-suite/README.md`.

Key Idea

Nothing from Weeks 1–3 was removed or rewritten. Everything below is *additive* — the running examples you already saw (`calculate_late_fee`, the shopping cart, discounts, `determine_shipping_cost`) are all still there, unchanged, and still behave the same way, including their two known, intentional `xfail`/FAIL results.

2 What Changed At a Glance

File	Status	Purpose
<code>app/notifications/password_reset.py</code>	New	A direct transcription of the reading's <code>PasswordResetService</code> / <code>smtplib</code> worked example from “Dependency Isolation and Seams”: a dependency-injection seam (<code>email_client</code>) replacing a hard-coded dependency.
<code>tests/notifications/test_password_reset_service.py</code>	New	On <code>password_reset.py</code> , proving the injected seam works, plus the reading's “mocks that lie” pitfall, as three runnable <code>pytest</code> tests.
<code>app/orders/order_service.py</code>	New	A direct transcription of the reading's separate “Test Double Taxonomy” worked example: an order-placement flow with three collaborators (payment, database, email).
<code>tests/orders/test_order_service.py</code>	New	The reading's full dummy/stub/fake/spy/-mock taxonomy, as six runnable <code>pytest</code> tests, one per collaborator scenario the reading walks through.
<code>app/accounts/registration_service.py</code>	New	<code>UserRegistrationService</code> : the first caller of <code>UserRepository</code> that needs isolating — fulfills that file's own Week 2 docstring forward-reference to a future mocking lecture.
<code>tests/accounts/test_registration_service.py</code>	New	Mocks <code>UserRepository</code> directly (via <code>create_autospec</code>) instead of a real <code>sqlite3</code> connection, plus a second “mocks that lie” demonstration.
<code>docs/DEMO_GUIDE_week04_tdd_test_doubles.md</code>	New	A section-by-section lecture companion (primarily for in-class use).
<code>README.md</code>	Modified	Updated project layout description and the expected <code>pytest -v</code> output, now that Week 4's files exist.

Key Idea

The reading updated its “Test Double Taxonomy” section after this codebase's first Week 4 pass, switching from illustrating all five doubles against `PasswordResetService` to a separate order-placement example. This revision follows that update: the seam worked example (`password_reset.py`) and the taxonomy worked example (`order_service.py`) are now two distinct files, matching the reading's own two distinct sections.

Note that `tests/practice/tdd_password_demo.py` (the Red-Green-Refactor live-coding script for the reading's `is_strong_password()` example) was already added to the repository ahead of this reading, in anticipation of this lecture. It is unchanged by this update and is described here for completeness only.

3 New App Code: PasswordResetService

Location: `app/notifications/password_reset.py`

This file is the centerpiece worked example from the reading’s “Dependency Isolation and Seams” section, transcribed so it can be run against real code instead of a slide. It defines three things:

- **EmailClient** — an interface with no real behavior. Its only job is to give `unittest.mock.create_autospec` something concrete to constrain a mock to (see the Common Pitfalls tests, below).
- **SmtplibEmailClient** — the real, production implementation, wrapping `smtplib`. It is intentionally *not* exercised by any test in this suite — testing it for real would require a real mail server, which is exactly the problem dependency injection solves.
- **PasswordResetService** — the class actually under test. Its constructor accepts `email_client` and `logger` from outside instead of constructing them internally.

Key Idea

The reading’s “before” version — a hard-coded `smtplib.SMTP(...)` call built directly inside `send_reset_email()` — is described in this file’s docstring but deliberately *not* reproduced as runnable code. Only the “after” version, with the seam already in place, actually exists here. This mirrors how Week 3’s `shipping.py` carried only the specification, not a second broken copy of the function.

4 New Tests: Proving the Seam Works

Location: `tests/notifications/test_password_reset_service.py`

Since the reading’s Test Double Taxonomy no longer uses `PasswordResetService`, this file keeps only what the “Dependency Isolation and Seams” section itself calls for: one test proving the injected `email_client` seam works with “any object that has a `.send(to, message)` method,” exactly as the reading describes.

```
cd demo-testing-suite
pytest tests/notifications/test_password_reset_service.py::
    test_send_reset_email_uses_the_injected_client -v
```

The file also carries the reading’s “Mocks that lie” pitfall (Common Pitfalls, below), since `EmailClient` — defined in `password_reset.py` — is a convenient real interface to demonstrate it against.

5 New App Code and Tests: The Test Double Taxonomy

Locations: `app/orders/order_service.py`, `tests/orders/test_order_service.py`

The reading illustrates all five doubles with, in its own words, “a single running example: an `order_service` that, when placing an order, normally has to interact with three external collaborators — a payment service, a database, and an email service.” `order_service.py` transcribes that example’s three functions verbatim: `calculate_total(price, logger)`, `place_order(payment_service)`, and `register_user(email_service)`. `test_order_service.py` encodes all five doubles as six runnable tests, in the reading’s own order:

- **Dummy** — a plain `DummyLogger` against `calculate_total()`, plus `PoisonedDummyLogger` for the reading’s second Dummy tip box (any accidental use raises immediately).
- **Stub** — `PaymentStub`, feeding `place_order()`; no assertion is made about how `process_payment()` was called.
- **Fake** — `FakeDatabase`, an in-memory dictionary, verified via **state verification** (`fake_db.get_user(...)` after the fact).
- **Spy** — `EmailSpy`, feeding `register_user()`; records `sent_to` for the test to inspect afterward.
- **Mock** — a bare `Mock()`, feeding `place_order()` again, verified via **behavior verification** (`assert_called_once_with(100)`).

Tip

Run the suite yourself:

```
cd demo-testing-suite
pytest tests/orders/test_order_service.py -v
```

All six tests pass. Unlike Weeks 1 and 3, there is no deliberate bug to find in this file — the teaching payload is in reading the five doubles side by side, not in a failure. Notice that Stub and Mock both drive the *same* `place_order()` function; only what the test checks afterward differs — the reading’s own way of distinguishing the two.

6 New App Code and Tests: Mocking UserRepository

Locations: `app/accounts/registration_service.py`, `tests/accounts/test_registration_service.py`

`app/accounts/user_repository.py`’s docstring has said, since Week 2: “A *mocking lecture can mock UserRepository instead of sqlite3 itself*.” Until now, nothing in the codebase actually called `UserRepository`, so that sentence had nothing to point at. `UserRegistrationService` is that first caller: it accepts a `user_repository` and an `email_client` via constructor injection, registers a new user through the repository, and welcomes them through the email client.

Its test file never touches `sqlite3`. Instead, `create_autospec(UserRepository, instance=True)` replaces the whole dependency, and the tests check both outcomes (a returned user id, a rejected duplicate) and interactions (the welcome email’s exact arguments, or that no email is sent on rejection). A third test repeats the “mocks that lie” point from the notifications suite, this time against `UserRepository`’s own interface.

Tip

Contrast this with `tests/accounts/test_user_repository.py`, which still exercises a real `:memory:` SQLite connection — that file tests `UserRepository` itself, while this new file tests a *caller* of `UserRepository` in isolation from it.

```
pytest tests/accounts/test_registration_service.py -v
```

7 New Documentation

Location: docs/DEMO_GUIDE_week04_tdd_test_doubles.md

This is a lecture-companion document, primarily written to guide what gets shown and run during class, section by section, following the Week 4 reading’s own structure — including how to live-code `tests/practice/tdd_password_demo.py`’s Red-Green-Refactor cycles. You are welcome to read it, but it is not required reading — the reading itself is the primary material.

8 Updated: README.md

The top-level `demo-testing-suite/README.md` was updated to reflect the six new app/test files above and to describe the suite’s current expected output. If you run the full suite from the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
pytest -v
```

you should now see **45 passed, 2 xfailed, 1 failed** — twelve more passing tests than Week 3’s baseline, and the same two xfail results and one FAIL as before, all unrelated to this week’s additions.

9 How This Connects to the Reading

Key Idea

The reading argues that TDD’s real value is design pressure: writing a test first surfaces hidden dependencies before they calcify. Dependency injection turns a hidden dependency into a seam, and the dummy/stub/ fake/spy/mock vocabulary gives a precise way to choose what belongs on the other side of that seam during a test. `app/notifications/password_reset.py`, `app/orders/order_service.py`, and `app/accounts/registration_service.py` exist so you can see all three pieces of that argument against real, running code: a seam introduced where a hard-coded dependency used to be, all five kinds of double filling seams in five different tests, and a real caller mocked in isolation from its dependency.

10 Suggested Next Steps

1. Read `app/notifications/password_reset.py`’s docstring and identify, in your own words, why the reading’s “before” version would be hard to unit test — before looking at how the “after” version solves it.
2. Run `pytest tests/orders/test_order_service.py -v` and, for each of the five doubles, say out loud which school (classical or mockist) it belongs to and why.
3. Run `pytest tests/accounts/test_registration_service.py -v` and explain why this file needs no real database at all, in contrast to `tests/accounts/test_user_repository.py`.

4. Try Exercise 2 from the reading (`SessionManager` / `time.time()`) on paper: identify the hard-coded dependency and sketch the constructor-injected version, the same way this week's app code did for `PasswordResetService` and `UserRegistrationService`.